National University of Computer and Emerging Sciences



Laboratory Manual 01

for

Computer Networks Lab

(CL -3001)

Course Instructor	Dr. Abdul Qadeer
Lab Instructor	Ms. Halima Sadia
Section	BCS-5A
Semester	Fall 2026

Department of Computer Science
FAST-NU, Lahore, Pakistan

Part: 01

Why Learn Python Before Computer Networks?

Before diving into Computer Networks (sockets, protocols, packet analysis, and network automation), it is essential to be comfortable with Python. Networking courses and tools increasingly rely on Python as the glue language that lets you script, test, and automate everything from a simple ping sweep to a full protocol implementation.

- Readable syntax: Python's clean, English-like syntax lets you focus on networking logic (sockets, protocols, packet structure) instead of fighting compiler errors.
- Rapid prototyping: You can write and test a working TCP/UDP client-server pair in a few lines — ideal for understanding protocols hands-on.
- Industry-standard tooling: Libraries such as socket, scapy, requests, and paramiko are used daily for network programming, packet crafting, and automation.
- Automation & scripting: Real network engineers use Python to automate device configuration (e.g., with Netmiko/Nornir) instead of manual CLI work.
- Strong foundation: The data structures covered in this handout — lists, tuples, dictionaries — are exactly what you will use to store IP addresses, routing tables, packet headers, and configuration data in later networking assignments.

In short, Python fundamentals now will make every Computer Networks lab (sockets programming, client-server models, protocol simulation) dramatically easier later.

Online Python Interpreter: Google Colab

For this Lab, we will use **Google Colab** to write and run our Python programs. Colab lets you write and execute Python directly in your browser, no installation required, and saves your notebooks automatically to Google Drive, where you can share them with others to view, comment, or edit.

Colab notebooks are Jupyter notebooks, combining live code, text, images, and visualizations in one document. They're widely used for data analysis, simulations, visualization, and machine learning.

Getting Started

1. Go to https://colab.research.google.com/ and sign in with your FAST-NU email.
2. Go to **File → New Notebook**.
3. Rename the notebook (default: Untitled0.ipynb) to your roll number.
4. In the first code cell, type:

```
print('Hello World')
```

5. Run the cell by clicking the play button or pressing **Ctrl + Enter**.
6. The notebook will connect to a runtime, allocating RAM and disk for your session

1. Python Basics

Objectives:

- Python data types

- Python operators (math, comparison, boolean)
- Python conditions and loops
- Python functions

1.1 Data Types

Python's built-in data types are grouped into categories: numeric, text, boolean, sequence, mapping, set, and binary types. Python has no explicit 'declare a variable' command, a variable is created the moment a value is first assigned to it, and variable names are case-sensitive.

Category	Data Type	Examples
Numeric	int	-2, -1, 0, 1, 2, int(20)
Numeric	float	-1.25, 0.0, 1.25, float(20.5)
Numeric	complex	1j, complex(1j)
Text Sequence	str	'a', 'Hello!', str("Hello")
Boolean	bool	True, False, bool(5)
Sequence	list	["apple", "banana", "cherry"]
Sequence	tuple	("apple", "banana", "cherry")
Sequence	range	range(6)
Mapping	dict	{"name": "John", "age": 36}
Set	set	{"apple", "banana", "cherry"}
Set	frozenset	frozenset({"apple","banana"})
Binary	bytes	b"Hello", bytes(5)
Binary	bytearray	bytearray(5)

Data types can be identified using these built-in methods:

- *type(object)* returns the exact data type of an object for instance, type([1, 2]) returns list.
- *isinstance(object, classinfo)* checks if an object is an instance of a specified data type or class, returning True or False for instance isinstance(x, list) returns True

Example

```
port = 8080 # int
latency = 12.5 # float
host = "192.168.1.1" # str
is_active = True # bool
```

```
# Typecasting: converting one data type into another
port_as_text = str(port) # int -> str
text_as_int = int("1024") # str -> int
print(type(port), type(port_as_text))
```

1.2 Operators

Operator	Operation	Example
**	Exponent	2 ** 3 = 8
%	Modulus / Remainder	22 % 8 = 6
//	Integer Division	22 // 8 = 2
/	Division	22 / 8 = 2.75
*	Multiplication	3 * 3 = 9
-	Subtraction	5 - 2 = 3
+	Addition	2 + 2 = 4

Comparison Operators:

Operator	Meaning
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

Example

```
packet_size = 22
n = 8
print(packet_size // n) # 2 (integer division)
print(packet_size % n) # 6 (remainder)

is_reachable = True
```

```
has_timeout = False
print(is_reachable and not has_timeout) # True
```

1.3 Conditions (if / elif / else)

Python supports if, elif, and else. Only if is required; elif and else are optional. Python uses indentation (not curly braces) to define code blocks.

Example:

```
status_code = 404
if status_code == 200:
 print("OK")
elif status_code == 404:
 print("Not Found")
elif status_code >= 500:
 print("Server Error")
else:
 print("Unknown status")
```

1.4 Loops

Python has two loop types: while and for.

Example **(while loop with break)**

```
attempts = 0
while True:
 attempts += 1
 print("Pinging... attempt", attempts)
 if attempts == 3:
 break # stop once we reach 3 attempts
```

Example **(while loop with continue)**

```
n = 0
while n < 5:
 n += 1
 if n == 3:
 continue # skip printing 3
 print(n)
```

Output: 1 2 4 5

Example **(for loop with range(start, stop, step))**

```
for port in range(20, 26, 2):
```

```
print("Scanning port:", port)
```

Output: Scanning port: 20, 22, 24

1.5 Functions

Functions start with the def keyword, followed by the function name and parentheses. A return statement sends a value back to the caller.

Example

```
def greet(name):
 print("Hello,", name)

greet("Fatima") # Hello, Fatima
```

***Example* (function with a return statement)**

```
def is_valid_port(port):
 return 0 <= port <= 65535

print(is_valid_port(8080)) # True
print(is_valid_port(70000)) # False
```

Built-in functions such as print(), len(), type(), int(), range(), and sum() are always available without needing an import.

2. Python Lists

Everything in Python is an object. Lists represent ordered, mutable sequences of values — they can be modified in place. Individual elements are accessed with square brackets, using zero-based indexing (the first element is index 0).

Creating lists

```
planets = ["Mercury", "Venus", "Earth", "Mars"]
empty_list = []
mixed = [1, "two", 3.0, True]
```

2.1 Indexing & Slicing Examples

Using the planets list

```
print(planets[0]) # Mercury (first element)
print(planets[-1]) # Mars (last element)
print(planets[1:3]) # ['Venus', 'Earth']
print(planets[:2]) # ['Mercury', 'Venus']
print(planets[::-1]) # reversed list
```

2.2 List Modification Examples

Modifying the planets list

```
planets[0] = "Mercury I" # update by index
planets.append("Jupiter") # add to the end
planets.insert(1, "Pluto") # insert at index 1
planets.remove("Pluto") # remove first match
del planets[0] # delete by index
```

2.3 List Functions

Function / Method	Purpose
len(list)	Number of elements
list.sort()	Sort in place
sorted(list)	Return a new sorted list
list.reverse()	Reverse in place
list.count(x)	Count occurrences of x
list.index(x)	First index of x
min(list) / max(list)	Smallest / largest value
sum(list)	Sum of numeric elements

Example

```
scores = [88, 95, 72, 95, 60]
print(len(scores)) # 5
print(max(scores)) # 95
print(scores.count(95))# 2
scores.sort()
print(scores) # [60, 72, 88, 95, 95]
```

2.4 List Comprehensions

List comprehensions are one of Python's most distinctive features. Combined with functions like min, max, and sum, they let you write in one line what would otherwise take several.

Example

```
squares = [n ** 2 for n in range(1, 6)]
print(squares) # [1, 4, 9, 16, 25]


even_ports = [p for p in range(20, 30) if p % 2 == 0]
print(even_ports) # [20, 22, 24, 26, 28]
```

3. Python Tuples

Tuples are almost exactly like lists, with two key differences: they are created with parentheses instead of square brackets, and they are immutable; once created, they cannot be modified. Tuples are often used for functions with multiple return values.

Creating and using tuples

```
coordinates = (33.6844, 73.0479) # (latitude, longitude)
print(coordinates[0]) # 33.6844
def min_max(values):
 return min(values), max(values) # returns a tuple
 lo, hi = min_max([4, 9, 1, 7])
print(lo, hi) # 1 9
```

3.1 Tuple Functions

A tuple object only supports two methods: count() and index(), since tuples are immutable, there are no methods that modify them.

Example

```
grades = ("A", "B", "A", "C", "A")
print(grades.count("A")) # 3
print(grades.index("C")) # 3
```

4. Python Dictionaries

Dictionaries and lists share the following characteristics:

- Both are mutable.
- Both are dynamic(they can grow and shrink as needed)
- Both can be nested (a list can hold a list; a dict can hold a dict or a list, and vice versa).

Dictionaries differ from lists in how elements are accessed:

- List elements are accessed by position (index).
- Dictionary elements are accessed by key, not by numerical index.

Duplicate keys are not allowed, and a dictionary key must be an immutable type, a key cannot be a list or another dict.

Creating dictionaries

```
device = {"name": "Router-1", "ip": "192.168.1.1", "active": True}
empty_dict = {}
device2 = dict(name="Switch-1", ip="192.168.1.2")
```

4.1 Dictionary Modification Examples

Accessing, adding, and updating entries

```
print(device["ip"]) # 192.168.1.1
device["location"] = "Lab 3" # add a new key-value pair
device["ip"] = "192.168.1.10" # update an existing value
device.pop("active") # remove a key
print("ip" in device) # True
```

4.2 Dictionary Formatting Example

The % operator can conveniently substitute values from a dictionary into a string by name. Modern code more commonly uses f-strings, shown alongside it below.

String formatting with a dictionary

```
info = {"name": "Router-1", "ip": "192.168.1.1"}

print("%(name)s is at %(ip)s" % info) # classic % formatting
print(f"{info['name']} is at {info['ip']}") # f-string (recommended)
```

Output: Router-1 is at 192.168.1.1

4.3 Dictionary Functions

Function / Method	Purpose
dict.keys()	All keys in the dictionary
dict.values()	All values in the dictionary
dict.items()	All (key, value) pairs
dict.get(key, default)	Safe lookup with a fallback
dict.update(other)	Merge another dict in
dict.pop(key)	Remove a key and return its value
len(dict)	Number of key-value pairs

Example

```
device = {"name": "Router-1", "ip": "192.168.1.1", "location": "Lab 3"}
for key, value in device.items():
```

```
print(key, "->", value)
# Dictionary comprehension
lengths = {k: len(str(v)) for k, v in device.items()}
```

For more details, visit iterate dictionary & dictionary comprehensions

5. Python Sets

A set is an unordered, unindexed collection of *unique* elements. Duplicate items are automatically removed. Sets are mutable, but their elements must be immutable types (strings, numbers, tuples).

Creating sets

```
ports = {80, 443, 80, 22}
print(ports) # Output: {80, 22, 443} (duplicates automatically removed)
```

Set operations (useful for IP/network comparisons)

```
active_ips = {"192.168.1.1", "192.168.1.2"}
scanned_ips = {"192.168.1.2", "192.168.1.3"}
# Common Set Methods
print(active_ips.union(scanned_ips)) # All unique IPs across both sets
print(active_ips.intersection(scanned_ips)) # IPs present in both sets
print(active_ips.difference(scanned_ips)) # IPs in active but not in scanned
```

Lab Tasks

In Lab Statements: (15)

1. Weighted Grade Calculator (01)

A student has three scores: quiz (20% weight), midterm (30% weight), and final (50% weight). Write a function final_grade(quiz, midterm, final) that computes the weighted percentage using arithmetic operators, then assigns a letter grade using if/elif/else (A ≥ 90, B ≥ 80, C ≥ 70, D ≥ 60, else F). Add one extra rule: regardless of the weighted percentage, if any individual score is below 40, the function must return "F". Think carefully about where this check belongs relative to your other conditions.

Input: final_grade(95, 88, 92)
Output: "A"
Input: final_grade(95, 88, 35)
Output: "F"

2. Armstrong Port Validator (02)

A 3-digit Armstrong number is one where the sum of the cubes of its digits equals the number itself (for example, 153 = $1^3 + 5^3 + 3^3$). Write a function is_armstrong(n) that determines this using only the // and % operators to extract digits (no converting the number to a string). Then write find_armstrong_ports(start, end) that loops through a range and returns a list of all Armstrong numbers found.

Input: is_armstrong(153)
Output: True
Input: find_armstrong_ports(100, 999)
Output: [153, 370, 371, 407]

3. Password Strength Auditor (02)

Write a function password_strength(password) that loops through each character of the string and, using only comparisons (no imports) determines whether the password contains at least one uppercase, one lowercase letter, one digit and one character from the set !@#$%^&* and whether its length is at least 8. The function should return a score from 0 to 5 (one point per satisfied rule) along with a label: 0-2 = "Weak", 3-4 = "Moderate", 5 = "Strong".

Input: password_strength("fast123")
Output: (2, "Moderate")
Input: password_strength("FastNU@2026")
Output: (5, "Strong")

4. IP Address Validator (02)

Write a function that checks if a given string is a valid IPv4 address (four dot-separated numbers, each between 0 and 255, no leading zeros like "01"). Don't use any import, do it with string splitting, loops, and conditions only.

Input: "192.168.1.1" → True
Input: "256.100.50.0" → False
Input: "192.168.01.1" → False
Input: "192.168.1" → False

5. Palindrome Detection (01)

Given a string s, create a function to detect if it's a palindrome, ignoring spaces, punctuation, and letter case. A palindrome is a word, phrase, number, or other sequence of characters that reads the same backward or forward.

Input: "A man a plan a canal Panama" → True
Input: "Race a Car" → False

6. List Flattening (02)

Given a nested list of integers, create a function to flatten the list to a single-level list. For example, [[1, 2, 3], [4, 5]] will be flattened to [1, 2, 3, 4, 5] and [[0, 4, 7], [3, 5],11] to [0, 4, 7, 3, 5,11].

7. Unique values in a dictionary (01)

Given a dictionary, return a list of all the values in the dictionary, but without any duplicates. Preserve the order in which value first appears.

Input: {"a": 1, "b": 2, "c": 1, "d": 3, "e": 2, "f": 1}
Output: [1, 2, 3]

8. Maximum and Minimum in List of Tuples (02)

Write a Python function to find the tuple with the maximum and minimum values based on the last element of each tuple.
Input: [(1, 2), (3, 4), (5, 0)]

Output: Min = (5, 0), Max = (3, 4)

Input: [(1, 2), (3, 4), (5, 0), (4, 4), (1, 0)]
Output: Min = (5, 0), (1, 0), Max = (3, 4), (4, 4)

9. Matrix Transpose (02)

Given a 2D list (matrix) of integers, create a function to return its transpose (rows become columns). Input: [[1, 2, 3], [4, 5, 6]]
Output: [[1, 4], [2, 5], [3, 6]]

Part: 02

SSH Lab Manual: Remote Login & Secure File Transfer

What is SSH, and Why Do We Need It?

A packet sniffer sitting on a shared network can capture a copy of every packet that passes by. Older remote access tools such as Telnet (for remote login) and FTP (for file transfer), send everything, including passwords, as plain, readable text. Anyone sniffing the network can read those credentials directly.
SSH (Secure Shell) was built to solve exactly this problem. It is a protocol that lets one machine (the client) securely log into and control another machine (the server) over a network, with the entire session encrypted.

SSH provides three guarantees:

- **Confidentiality**: all traffic is encrypted, unreadable to a sniffer.
- **Integrity**: any tampering with the data in transit is detectable.
- **Authentication**: the client verifies it is really talking to the intended server, and the server verifies the identity of the client.

SSH replaced two older insecure tools at once:

	Old insecure tool	Secure replacement
Remote login / shell	Telnet	SSH
File transfer	FTP	SCP / SFTP (run over SSH)

Every SSH connection has two roles:

- **SSH Server (sshd)**: the machine being connected into. It must be running an SSH server process, listening for connections (default port 22).
- **SSH Client (ssh)**: the machine connecting out, initiating the login.

Checking the SSH Client is Available(client-machine):

Almost every modern OS ships with an SSH client pre-installed. Confirm this first:

```
fatima@fastlhr:~$ ssh -V
OpenSSH_9.6p1 Ubuntu-3ubuntu13.13, OpenSSL 3.0.13 30 Jan 2024
```

A version string such as OpenSSH_9.x confirms the client is ready to use.

Starting the SSH Server(on server-machine)

On Linux (Ubuntu):

sudo apt update

sudo apt install openssh-server -y

sudo systemctl enable ssh

sudo systemctl start ssh

sudo systemctl status ssh

```
fatima@fastlhr:~$ sudo apt update
sudo apt install openssh-server -y
sudo systemctl enable ssh
sudo systemctl start ssh
sudo systemctl status ssh   # confirm it's "active (running)"
[sudo] password for fatima:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [967
kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [205
```

This means the server is actively listening for incoming connections on all available network addresses. A connection additionally requires network reachability (same LAN) and valid credentials (a real username plus a correct password or authorized key) before any access is granted.

Remote Login (Client Side Connection)

To initiate a remote terminal session, use the standard connection syntax(run this command on client machine): *ssh username@server_ip* (username and ip will be of server machine)

```
fatima@fastlhr:~$ ssh fatima@192.168.196.128
The authenticity of host '192.168.196.128 (192.168.196.128)' can't be establishe
d.
ED25519 key fingerprint is SHA256:g5RRHepnGyvuv+mhwLLIXZoQ8oLbkhzIpHxAxA/sXzs.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

First-time Connection Verification:

When connecting to a server for the first time, SSH prompts you to verify the host key fingerprint to prevent man-in-the-middle attacks:

Type 'yes' and press Enter. The server's key will be stored in your local '~/.ssh/known_hosts' file, and you will be prompted for the user password.

Before typing yes, run below command on server machine to see its fingerprint and match it with one you get on client side and enter yes.

ssh-keygen -l -f /etc/ssh/ssh_host_ed25519_key.pub

Secure File Copying (SCP & SFTP)

To securely transfer files over the encrypted SSH tunnel, use SCP (Secure Copy Protocol) or SFTP (SSH File Transfer Protocol).

1. Copying Files with SCP

To push a local file to the remote server:
scp local_file.txt username@server_ip:/path/to/remote/dir/

To pull a file from the remote server to your local directory:
scp username@server_ip:/path/to/remote/dir/file.txt ./

To transfer an entire directory recursively, use the -r flag:

scp -r ./my_folder username@server_ip:/home/username/

2. Interactive Transfers with SFTP

For an interactive session to explore, upload, and download files:

sftp username@server_ip

Common SFTP commands inside the interactive shell:
• **ls / cd**: list files and change directories on the remote server.
• **lls / lcd**: list files and change directories on the local machine.
• **put filename**: upload a file from local machine to remote server.
• **get filename**: download a file from remote server to local machine.
• **exit** : terminate the SFTP session.

Hands-On Exercise (10)

Phase 1: Student A (Server) & Student B (Client)
Student A enables the SSH server and shares their IP address. Student B initiates an SSH connection, verifies Student A's host key fingerprint directly against the server's public key before accepting, and logs in. Student B runs commands on Student A's machine to generate a system report containing the host specifications, current timestamp, and disk usage. After logging out, Student B connects via interactive **SFTP** to pull the report to their local machine and inspect its contents.
Phase 2: Student B (Server) & Student A (Client)

- Swap the roles. Student B hosts the server and Student A connects via SSH after completing the host fingerprint verification. On Student B's system, Student A creates a project folder containing an executable bash script designed to print a custom greeting.
- Student A exits SSH, launches an interactive SFTP session to download the script from Student B's machine, and executes the pulled script locally to confirm file integrity and execution permissions.

Submission guidelines:

• Part 1 (Python Revision Tasks): Complete Part 1 within the first hour using Google Colab. • Part 2: Work on Part 2 in the Ubuntu environment and complete all assigned tasks. • For Part

1, submit one .ipynb file containing all the completed tasks.
• For Part 2, submit one Word file containing screenshots of the output for all completed tasks. • Make sure your files are properly organized and submitted on Google Classroom within the given deadline.